AlphaDetect

Explainable MIMO Detection Algorithm Discovery via Neural-Symbolic Automated Reasoning

Research Proposal

April 2026

Abstract

We propose **AlphaDetect**, a unified framework for the automated discovery of explainable MIMO detection algorithms. AlphaDetect grounds algorithm discovery in *formal reasoning* through four tightly integrated components: (i) a Typed Domain-Specific Language (DSL) that encodes signal-processing and probabilistic-inference primitives with precise mathematical types and algorithmic combinators; (ii) a Problem Transformation Library that recasts the invention of intermediate concepts as witness generation from typed transformations; (iii) a Computer Algebra System (CAS) that performs step-by-step symbolic derivation of algorithm update equations; and (iv) a Structure Mapping Engine (SME) augmented with LLM-generated predicate representations for cross-domain conceptual transfer. Together, these components address the central challenge of algorithm discovery—the invention of novel *intermediate concepts* (e.g., recognising a MIMO model as a factor graph, or introducing a cavity distribution) that cannot be reduced to search over a fixed concept space. We present the complete system architecture, detail every component and its implementation, and demonstrate through case studies (EP-MIMO, GTA, BP-guided sphere decoding) how known algorithms can be re-derived within this framework.

This document describes the AlphaDetect research project in full detail.

Contents

1 Introduction and Motivation **3**
1.1 The MIMO Detection Problem . . . 3
1.2 The Algorithm Discovery Gap . . . 3
1.3 Why Standard Approaches Fall Short . . . 3
1.4 Research Vision . . . 3

2 The Conceptual Innovation Problem **4**
2.1 What Is a "Concept" in Algorithm Discovery? . . . 4
2.2 Algorithmic Combination as a Special Case . . . 4

3 System Architecture **5**

4 Typed Domain-Specific Language **6**
4.1 Design Philosophy . . . 6
4.2 Type Hierarchy . . . 6
4.3 Primitive Operations . . . 6
4.4 Algorithmic Combinators . . . 7
4.5 Type-Adapter Library . . . 7

5 Problem Transformation Library **8**
5.1 Transformations as Concept Generators . . . 8
5.2 Searchability of the Transformation Space . . . 8

6 CAS-Driven Derivation Engine **8**
6.1 Role of the CAS . . . 8
6.2 CAS Implementation Details . . . 9

7 LLM-Augmented Structure Mapping Engine **9**
7.1 Structure Mapping Theory . . . 9
7.2 The Representation Bottleneck and LLM Resolution . . . 10
7.3 SME Matching and Candidate Inferences . . . 10
7.4 LLM-Augmented Semantic Matching . . . 11
7.5 LLM as Source-Domain Recommender . . . 11

8 Search Engine and Evaluation **11**
8.1 Search Strategy . . . 11
8.2 Complexity Evaluator . . . 11
8.3 Performance Oracle . . . 11
8.4 Concept Space Self-Enlargement . . . 12

9 Case Studies **12**
9.1 Case Study 1: Automatic Derivation of EP-MIMO . . . 12
9.2 Case Study 2: Gaussian Tree Approximation (GTA) . . . 13
9.3 Case Study 3: BP-Guided Sphere Decoding . . . 14
9.4 Discoverability Comparison . . . 15

10 Discussion: Scope and Limitations **15**
10.1 DSL Granularity Trade-off 15
10.2 The Irreducible Role of Structural Intuition 15
10.3 Axiomatic Probing as a Semantic Fallback 15

11 Related Work **16**

12 Summary and Contributions **16**

1. Introduction and Motivation

1.1. The MIMO Detection Problem. Massive MIMO systems are at the heart of 5G and beyond. Given the received signal model

$$\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}, \qquad \mathbf{n} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I}), \tag{1}$$

where $\mathbf{H} \in \mathbb{C}^{N_r \times N_t}$ is the channel matrix, $\mathbf{x}$ is the transmitted symbol vector drawn from a finite constellation $\mathcal{S}$, and $\mathbf{y}$ is the received vector, the goal is to estimate $\mathbf{x}$ with minimum bit-error rate (BER) under polynomial computational complexity. Optimal maximum-likelihood detection is NP-hard for general constellations, motivating a large family of approximate algorithms.

1.2. The Algorithm Discovery Gap. State-of-the-art MIMO detectors—ZF, MMSE, V-BLAST, sphere decoding, KBEST, BP on factor graphs, Gaussian Tree Approximation (GTA), Expectation Propagation (EP), AMP, and their many hybrids—were discovered by human researchers over decades of work. Each breakthrough required not just mathematical derivation, but the *invention of new concepts*: representing the detection problem as probabilistic inference on a graph, modelling the search space as a tree, approximating a loopy factor graph with a Chow-Liu spanning tree, or introducing the notion of a cavity distribution.

Core Challenge

Can a machine systematically discover such *conceptual innovations*, rather than merely optimising within a fixed algorithm space?

1.3. Why Standard Approaches Fall Short.

Neural architecture search (NAS) and deep unrolling. These learn parameters of a fixed computational graph. They do not discover new algorithmic *structures*.

LLM-based algorithm evolution. Approaches such as FunSearch or AlphaCode evolve programs by sampling from an LLM. We explicitly exclude these: the discovered programs are opaque and cannot be formally verified.

Classical Neural-Symbolic AI. Existing systems excel at problems with known rules and a fixed solution space (e.g., Sudoku, theorem proving from axioms). Algorithm discovery is different: there is no canonical target, and the solution space—the set of useful algorithms—is itself unknown.

1.4. Research Vision. AlphaDetect targets **explainable** algorithm discovery. Every step in a discovered algorithm must be traceable to a formal derivation, a type-correct composition of primitives, or a verified algebraic manipulation. The system is built upon

four tightly integrated components that jointly cover the full spectrum of algorithmic innovation:

1. A **Typed DSL** that encodes algorithm primitives with precise mathematical types, complexity annotations, and higher-order combinators.
2. A **Problem Transformation Library** that formalises the "invention of intermediate variables" as the application of typed transformations to sub-problems.
3. A **CAS-driven Derivation Engine** that symbolically derives algorithm update rules once the high-level structure is fixed.
4. An **LLM-augmented Structure Mapping Engine** that discovers cross-domain analogies and proposes new conceptual hypotheses.

2. The Conceptual Innovation Problem

2.1. What Is a "Concept" in Algorithm Discovery?. When a researcher proposes modelling MIMO detection as inference on a factor graph, she is not applying a known rule—she is *expanding the concept space* itself. We characterise the three main forms of conceptual innovation encountered in MIMO algorithm discovery.

Innovation Type	Example	Difficulty
Structural analogy	MIMO posterior ↔ factor-graph inference	High (cross-domain)
Optimisation transformation	KL minimisation → EP via coordinate descent	Medium (CAS-derivable)
Algorithmic combination	BP beliefs guiding sphere-decoding search order	Low–medium (type-search)

Observation 2.1. *The invention of an intermediate variable is almost never a free creative act. It is the* witness *produced by applying a problem transformation. A QR decomposition of* $\mathbf{H}$ *produces* $\mathbf{Q}$ *and* $\mathbf{R}$ *as witnesses; a Lagrangian relaxation produces dual variables as witnesses; an approximate-distribution ansatz produces site approximations as witnesses.*

This observation is central to AlphaDetect: instead of inventing variables, the system *searches the transformation space*, and witnesses appear as by-products.

2.2. Algorithmic Combination as a Special Case. A large class of practically important innovations consists not of new concepts but of novel *combinations* of existing algorithms. Examples include BP-guided sphere decoding, OSIC (ordered SIC with MMSE ordering), iterative MMSE–BP, and message-passing-on-search-tree detectors. These can be discovered by searching the space of compositions of known algorithm modules using the typed combinators of the DSL (Section 4).

3. System Architecture

AlphaDetect is composed of four core modules—the Typed DSL, the Problem Transformation Library, the CAS Derivation Engine, and the LLM-augmented Structure Mapping Engine—coordinated by a Search Engine and evaluated by a Complexity Evaluator and Performance Oracle. Figure 1 illustrates the overall information flow.

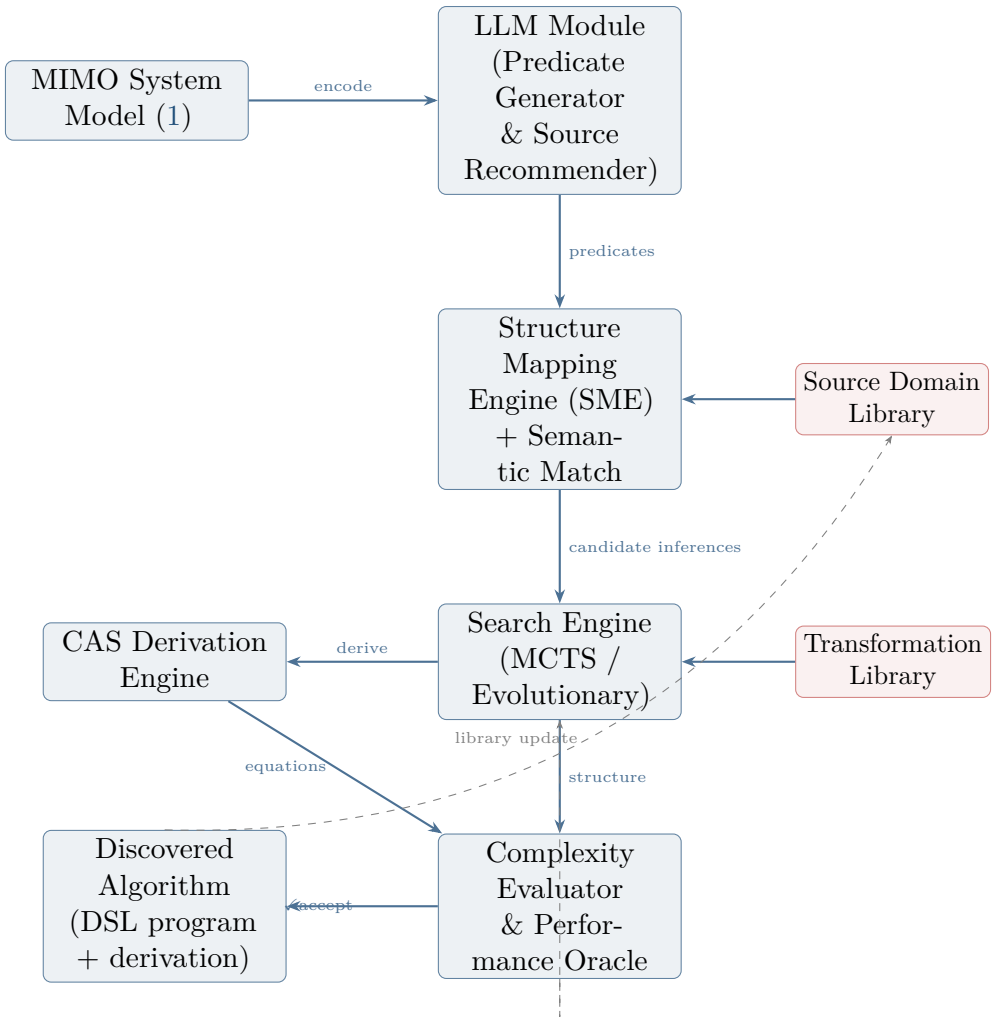

Figure 1: AlphaDetect system architecture. Solid arrows denote primary data flow; dashed arrows denote feedback (search pruning and library enrichment).

The system operates as follows. Given a MIMO system model, the LLM module encodes it as a predicate structure and recommends candidate source domains. The SME performs structural alignment between target and source domains, producing candidate inferences that hypothesise new conceptual structures (e.g., "the MIMO system can be represented as a factor graph"). The Search Engine then navigates the space of DSL compositions and problem transformations, guided by candidate inferences, type compatibility, and complexity constraints. When a high-level algorithmic structure is selected, the CAS derives the exact update equations. The resulting algorithm is evaluated for complexity and performance; successful discoveries are fed back to enrich the source-domain library

and DSL primitive set, enabling self-enlarging concept spaces.

4. Typed Domain-Specific Language

4.1. Design Philosophy. The DSL serves as the *semantic substrate* of AlphaDetect. Every expression in the DSL carries a precise type; every primitive comes with a complexity annotation; every combinator is typed, so the type system automatically enforces which combinations are legal without requiring semantic understanding of the involved concepts.

Key Insight

The type signature of a discovered intermediate object replaces human semantic understanding. If the system constructs an object of type `FactorGraph[Variable, Factor]`, all algorithms that accept a `FactorGraph` as input become immediately applicable—regardless of whether the system "understands" what a factor graph means.

4.2. Type Hierarchy. The DSL organises types in four layers.

Layer 0 — Mathematical foundations. Scalars, tensors, matrices with structural subtypes (Hermitian, PositiveDefinite, UpperTriangular, Unitary, etc.).

Layer 1 — Signal-processing objects. `MIMOSystem`, `Constellation`, `SoftEstimate`, `HardEstimate`, `LLR`, `Subspace`.

Layer 2 — Probabilistic / graph-model objects.

- `Distribution[X]` ⊃ `Gaussian[X]`, `DiscreteDistribution[X]`, `MixtureDist[X]`
- `Belief` ⊃ `GaussianBelief`, `DiscreteBelief`
- `BeliefSet[n]` ⊃ `GaussianBeliefSet[n]`, `DiscreteBeliefSet[n]`
- `Graph[Node,Edge]` ⊃ `Tree`, `BipartiteGraph` ⊃ `FactorGraph`, `WeightedGraph`
- `SearchTree[State]`, `CandidateList[T,Score]`

Layer 3 — Algorithm-control objects. `Ordering`, `Heuristic`, `Criterion`, `Complexity`.

4.3. Primitive Operations. Selected primitives illustrating the DSL's scope are listed in Table 1.

Table 1: Representative DSL primitives with type signatures and complexity.

Primitive	**Type Signature**	**Complexity**
`qr`	`Matrix`$[m, n] \rightarrow$ (`Unitary`$[m] \times$ `UpperTriangular`)	$O(mn^2)$
`mmse_equalize`	`MIMOSystem` $\rightarrow$ (`SoftEstimate`, `ErrorCov`)	$O(N_t^3)$
`system_to_factor_graph`	`MIMOSystem` $\rightarrow$ `FactorGraph`	$O(N_r N_t)$
`find_spanning_tree`	`WeightedGraph` $\rightarrow$ `Tree`	$O(E \log V)$
`gaussian_bp_message`	(`Factor`, `GaussianMessages`) $\rightarrow$ `GaussianMessage`	$O(N_t)$
`bp_message_factor_to_var`	(`Factor`, `Messages`) $\rightarrow$ `Message`	$O(M^d)$
`sic_detect_one`	(`MIMOSystem`, `Int`, `Partial`) $\rightarrow$ (`Symbol`, `MIMOSystem`)	$O(N_r N_t)$
`expand_node`	(`SearchTree`, `State`) $\rightarrow$ `List`[`State`, `Cost`]	$O(M)$
`select_top_k`	(`CandidateList`, k) $\rightarrow$ `CandidateList`	$O(n \log k)$
`mutual_info_weights`	(`GaussianBeliefSet`, `MIMOSystem`) $\rightarrow$ `WeightedGraph`	$O(N_t^2)$

4.4. Algorithmic Combinators. Combinators are *higher-order typed operators* that compose primitives without violating type safety.

```
sequence   : (Op[A->B], Op[B->C]) -> Op[A->C]
guide      : (Op[A x Heuristic -> B], Op[A -> Heuristic]) -> Op[A->B]
nest       : (IterOp[A->B], Op[SubProblem->SubResult]) -> Op[A->B]
iterate    : (Op[S->S], init:S, convergence:S*S->Bool) -> S
fold_layers : (LayerOp, Ordering[Nt], MIMOSystem) -> HardEstimate
refine     : (Op[A->B], Op[B*A->B], n_steps:Int) -> Op[A->B]
parallel_merge : (Op[A->B1], Op[A->B2], Op[(B1,B2)->C]) -> Op[A->C]
```

Listing 1: Core combinator signatures

4.5. Type-Adapter Library. Adapters bridge incompatible types and are the primary mechanism by which *cross-paradigm combinations* are discovered.

```
beliefs_to_ordering     : BeliefSet[n] -> Ordering[n]
beliefs_to_heuristic    : BeliefSet[n] -> Heuristic[PartialSymbolVec ->
    Score]
beliefs_to_candidates   : DiscreteBeliefSet[n] -> CandidateList[
   SymbolVec, Prob]
estimate_to_beliefs     : (SoftEstimate[n], ErrorCov) ->
   GaussianBeliefSet[n]
covariance_to_graph     : PositiveDefinite[n] -> WeightedGraph[n, Real]
partial_to_reduced_sys  : (MIMOSystem, PartialDecision[k]) ->
   MIMOSystem[Nt-k, Nr]
```

Listing 2: Key adapters bridging probabilistic beliefs and search control

The discovery of a new cross-paradigm algorithm reduces to finding a type-correct path through primitives, combinators, and adapters. The search engine enumerates candidate paths; the complexity evaluator prunes infeasible ones; performance on a benchmark channel model selects among survivors.

5. Problem Transformation Library

5.1. Transformations as Concept Generators. The invention of an intermediate concept reduces to the *application of a problem transformation* whose output (the "witness") becomes the new object. The transformation library catalogues known patterns, organised by category.

Table 2: Problem transformation categories with typed signatures and witness objects.

Category	Transformation	Witness
Algebraic	`qr`: $\mathbf{H} \to \mathbf{QR}$	$\mathbf{Q}, \mathbf{R}$
	`woodbury`: $(A + UCV)^{-1} \to$ small inverse	interm. matrices
	`complete_square`: $f(x) \to g(x-h)^2 + k$	h, k
Probabilistic	`introduce_latent`: $p(\mathbf{x}) \to \int p(\mathbf{x}, \mathbf{z})\, d\mathbf{z}$	$\mathbf{z}$
	`factorize_joint`: $p(\mathbf{x}_{1:n}) \to \prod p(x_i \mid pa_i)$	graph, CPDs
	`exponential_family`: $p \to \exp(\theta^\top T(x) - A(\theta))$	θ, T
Optimisation	`lagrangian_relax`: $\min f$ s.t. $g \leq 0 \to \min_x \max_\lambda \mathcal{L}$	λ
	`coordinate_descent`: $\min_{x,y} f \to$ alternating mins	sub-objectives
	`surrogate_objective`: $f \to \tilde{f} \geq f$	surrogate, bound
Structural	`to_factor_graph`: system $\to$ `FactorGraph`	factor nodes
	`tree_approximate`: `Graph` $\to$ `Tree`	tree, approx err.

5.2. Searchability of the Transformation Space. Applying a transformation T to the current problem yields a witness and a transformed sub-problem. The sub-problem is matched against known solvable patterns (within the DSL type system). Two signals indicate a useful transformation:

- The transformed problem has a *lower complexity class*; or
- The transformed problem *type-matches* the input of a known efficient algorithm.

The search engine navigates the transformation space using a complexity-guided heuristic: transformations that reduce complexity class or increase structural constraint are preferred.

6. CAS-Driven Derivation Engine

6.1. Role of the CAS. Once the search engine has selected a high-level algorithmic structure (which primitives, which combinators, which transformation sequence), the CAS takes over to derive the exact update equations. The CAS does not invent concepts; it performs:

1. Symbolic expansion of objective functions (KL divergence, MSE, etc.)
2. Functional differentiation and stationarity conditions
3. Gaussian closure: marginalisation, conditioning, moment computation
4. Complexity class determination via symbolic asymptotic analysis

6.2. CAS Implementation Details. The CAS operates on symbolic expressions conforming to the DSL type system. Key capabilities include:

- **Gaussian algebra**: Given a joint Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, compute marginals, conditionals, and Schur complements in closed form.
- **Exponential-family calculus**: Given a distribution in exponential-family form $\exp(\theta^\top T(x) - A(\theta))$, compute natural gradients, sufficient statistics, and moment parameters.
- **KL divergence expansion**: Symbolically expand $\mathrm{KL}(q\|p)$ or $\mathrm{KL}(p\|q)$ for parametric families and derive stationarity conditions with respect to variational parameters.
- **Precision-domain arithmetic**: Maintain beliefs in precision (information) form to enable efficient cavity computations via subtraction rather than matrix inversion.
- **Asymptotic complexity analysis**: Given a symbolic expression, determine the asymptotic complexity class in N_t, N_r, and M.

7. LLM-Augmented Structure Mapping Engine

7.1. Structure Mapping Theory. Gentner's Structure Mapping Theory (SMT) [1] posits that analogy is not surface similarity but *structural alignment*: a bijective mapping between the relational structures of a source domain and a target domain. The three guiding principles are:

1. **Structural consistency**: the mapping is one-to-one.
2. **Systematicity principle**: higher-order relational structure (relations-among-relations) is preferred over isolated attribute matches.
3. **Candidate inferences**: predicates that exist in the source but have no target counterpart are *predicted* to potentially hold in the target—this is the mechanism for knowledge transfer.

The Structure Mapping Engine (SME) [2] implements SMT computationally, producing *gmaps* (globally consistent mappings) ranked by structural evaluation score.

7.2. The Representation Bottleneck and LLM Resolution. SME requires domains to be described as hand-crafted predicate structures. Chalmers et al. [5] criticised this: the creative work may reside in the representation design rather than in SME itself. AlphaDetect resolves this by using an LLM as an automated predicate generator.

A fine-tuned (or few-shot prompted) LLM encodes mathematical concepts as predicate structures conforming to the DSL's ontology:

```
;; Target domain: MIMO posterior inference
(coupled-variables x1 ... xNt H)
(intractable exact-marginal posterior)
(goal marginal-inference posterior x_i)
(additive-noise observation sigma-squared)
(constraint discrete-valued x constellation)
(complexity-target polynomial)
```

Listing 3: LLM-generated predicate encoding of the MIMO posterior inference problem

```
;; Source domain: Factor-graph probabilistic inference
(coupled-variables v1 ... vN factor-graph)
(factorized-distribution p factors)
(graph-structure factor-graph variables factors)
(message-passing bp factor-graph)
(tractable message-passing tree)
(intractable exact-message-passing general-graph)
(tree-decomposition general-graph tree)
(optimality Chow-Liu KL-optimal-tree-approximation)
```

Listing 4: LLM-generated predicate encoding of factor-graph inference (source domain)

7.3. SME Matching and Candidate Inferences. Running SME on the two predicate sets produces the following global mapping:

Source (factor-graph inference)	**Target (MIMO detection)**
`coupled-variables vi (factor-graph)`	`coupled-variables xi (H)`
`intractable exact-message-passing`	`intractable exact-marginal`
`goal marginal-inference`	`goal marginal-inference`

Candidate inferences (source predicates absent from target):

1. `(graph-structure ??? variables factors)` ⇒ *MIMO system can be represented as a factor graph.*
2. `(message-passing bp ??? → marginals)` ⇒ *BP can be applied to the MIMO factor graph.*
3. `(tree-decomposition ??? tree)` ⇒ *Approximate MIMO factor graph with a spanning tree.*

These three inferences, in sequence, correspond precisely to the conceptual steps that led human researchers to GTA.

7.4. LLM-Augmented Semantic Matching. Standard SME requires identical predicate names (functors) to produce a match. We replace this with LLM-embedding cosine similarity:

$$m(\pi_s, \pi_t) = \frac{\mathbf{e}_s^\top \mathbf{e}_t}{\|\mathbf{e}_s\|\|\mathbf{e}_t\|}, \qquad \mathbf{e} = \text{LLM_embed}(\text{functor} \oplus \text{args description}) \tag{2}$$

This allows SME to discover analogies across vocabularies (e.g. `propagate` ↔ `transmit`) and across domains encoded by different research communities.

7.5. LLM as Source-Domain Recommender. Given the target predicate set, the LLM recommends candidate source domains by ranking mathematical/engineering areas by estimated structural similarity. For MIMO detection, typical recommendations include: statistical physics (Ising inference), LDPC decoding, Markov random field labelling, lattice reduction, and convex optimisation. Each recommended domain is then encoded into predicates by the LLM, and SME computes formal Gmap scores.

8. Search Engine and Evaluation

8.1. Search Strategy. The Search Engine navigates the combined space of DSL compositions, problem transformations, and SME-suggested conceptual hypotheses. It employs two complementary strategies:

- **MCTS (Monte Carlo Tree Search)**: Each node in the search tree represents a partial algorithm (a sequence of DSL operations and transformations applied so far). Expansion adds one more primitive, combinator, or transformation. Rollout estimates performance via lightweight simulation on small MIMO instances. The UCB1 criterion balances exploration and exploitation.
- **Evolutionary search**: A population of candidate DSL programs evolves via mutation (replace one primitive), crossover (swap sub-expressions between two programs), and selection (retain programs with best performance-to-complexity ratio).

8.2. Complexity Evaluator. Every candidate algorithm is annotated with a symbolic complexity expression derived from the CAS. The evaluator gates the search by rejecting candidates whose complexity exceeds a user-specified budget (e.g., $O(N_t^3)$ for real-time detection).

8.3. Performance Oracle. Candidate algorithms that pass the complexity gate are evaluated on a benchmark suite comprising:

- Rayleigh i.i.d. channels ($N_t = N_r = 16, 32, 64$)
- Correlated channels (exponential correlation model)
- Ill-conditioned channels (condition number > 100)
- Multiple constellations (QPSK, 16-QAM, 64-QAM)

Performance is measured by BER at target SNR values. Algorithms that achieve BER improvements over the best known baseline for any scenario are flagged for human review.

8.4. Concept Space Self-Enlargement. A key design feature is the *self-enlarging concept space*:

1. Successful algorithm compositions are added to the source-domain library (SME) and as new primitives (DSL) via Library Learning [3].
2. The enriched libraries enable higher-order conceptual transfer that was impossible with the initial primitive set.

This mirrors the DreamCoder wake–abstraction–dream cycle, applied to algorithm discovery rather than program synthesis.

9. Case Studies

We now demonstrate, through three detailed case studies, how known MIMO detection algorithms can be re-derived within the AlphaDetect framework. Each case study traces the full path from problem specification to final algorithm, showing which components of the system are invoked at each step.

9.1. Case Study 1: Automatic Derivation of EP-MIMO. Expectation Propagation (EP) for MIMO detection [6] is an algorithm whose core innovations—the cavity distribution, the tilted distribution, and moment matching—are often presented as creative insights. We show that all these "concepts" emerge as algebraic by-products of the CAS derivation engine, given appropriate search decisions.

Setup. The Bayesian posterior is

$$p(\mathbf{x}|\mathbf{y}) \propto \underbrace{\mathcal{N}(\mathbf{y}; \mathbf{H}\mathbf{x}, \sigma^2\mathbf{I})}_{f_0(\mathbf{x})} \cdot \prod_{i=1}^{N_t} \underbrace{p(x_i)}_{f_i(x_i)}. \tag{3}$$

Exact marginalisation is $O(M^{N_t})$, flagged as infeasible by the complexity evaluator.

Search decision 1 — Approximate in Gaussian family. The search engine selects `approximate_in_family`(p, `Gaussian`, $\mathrm{KL}_{\mathrm{reverse}}$), producing witness $q(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

Search decision 2 — Match factorisation. The search engine applies `match_factorization`, yielding $q(\mathbf{x}) \propto f_0(\mathbf{x}) \cdot \prod_i \tilde{q}_i(x_i)$ with site approximations $\tilde{q}_i = \mathcal{N}(\mu_i, \sigma_i^2)$ as witnesses.

Search decision 3 — Coordinate descent. The search engine selects `coordinate_descent` over the $\tilde{q}_i$. The CAS then derives the optimal update for $\tilde{q}_i$ with all other factors fixed:

$$\underbrace{q_{-i}(\mathbf{x}) \triangleq \frac{q(\mathbf{x})}{\tilde{q}_i(x_i)}}_{\textit{cavity distribution (CAS witness)}} = \mathcal{N}(\boldsymbol{\mu}_{-i}, \boldsymbol{\Sigma}_{-i}) \tag{4}$$

$$\underbrace{\hat{p}_i(x_i) \propto f_i(x_i) \cdot q_{-i}^{\text{marg}}(x_i)}_{\textit{tilted distribution (CAS witness)}} = \sum_{s \in \mathcal{S}} w_s \, \delta(x_i - s), \quad w_s \propto \mathcal{N}(s; \mu_{-i}, \sigma_{-i}^2) \tag{5}$$

$$\tilde{q}_i^{\text{new}} : \quad \mathbb{E}_{\hat{p}_i}[x_i], \ \text{Var}_{\hat{p}_i}[x_i] \quad \leftarrow \quad O(M) \text{ computation} \tag{6}$$

The cavity parameters are obtained by precision-domain subtraction (CAS step):

$$\sigma_{-i}^{-2} = \sigma_{q,i}^{-2} - \sigma_{\tilde{q}_i}^{-2}, \qquad \mu_{-i}\sigma_{-i}^{-2} = \mu_{q,i}\sigma_{q,i}^{-2} - \mu_{\tilde{q}_i}\sigma_{\tilde{q}_i}^{-2}. \tag{7}$$

Key Insight

From the CAS perspective, *cavity distribution* and *tilted distribution* are merely intermediate algebraic quantities introduced to simplify expressions. Human researchers gave them names and intuitions; the CAS required neither.

Complexity verification. The full EP iteration has complexity $O(N_{\text{iter}} \cdot N_t \cdot (N_t^2 + M))$, which is polynomial—accepted by the evaluator.

Multi-path exploration. Different search decisions yield different algorithms:

KL direction	Factorisation	Family	Algorithm
KL($q\|p$)	Match prior	Gaussian	**EP-MIMO**
KL($q\|p$)	Mean field	Exponential family	Variational Bayes
KL($p\|q$)	Match prior	Gaussian	α-divergence approx.
–	–	Gaussian + Laplace	Laplace-EP

DSL representation of EP-MIMO.

$$\texttt{sites}_0 = \texttt{init_gaussian_sites}(N_t)$$
$$\hat{\mathbf{x}} = \texttt{iterate}\Big(\texttt{ep_update_step}(\mathit{sys}, \texttt{sites}),\ \texttt{sites}_0,\ \text{convergence}\Big)$$

where `ep_update_step` internally invokes the CAS-derived cavity computation, tilted distribution formation, and moment matching.

9.2. Case Study 2: Gaussian Tree Approximation (GTA). GTA [7] exemplifies a *structural innovation*: the key insight is to approximate the fully-connected MIMO factor graph with an optimal spanning tree. This case study demonstrates how the SME module drives discovery of structural concepts that the CAS alone cannot derive.

SME-driven hypothesis generation. Given the MIMO posterior inference problem as the target domain, the LLM recommends factor-graph inference as a candidate source domain. SME alignment produces the candidate inference: "the MIMO system can be represented as a factor graph, and the factor graph can be approximated by a spanning tree."

DSL program. The search engine, guided by the SME candidate inference, constructs:

$$\begin{aligned}
&\texttt{beliefs}_0 = \texttt{estimate_to_beliefs}(\texttt{mmse_equalize}(\mathit{sys}))\\
&\texttt{weights} = \texttt{mutual_info_weights}(\texttt{beliefs}_0,\ \mathit{sys})\\
&\texttt{tree} = \texttt{find_spanning_tree}(\texttt{weights},\ \mathit{objective} = \max)\\
&\hat{\mathbf{x}} = \texttt{iterate}(\texttt{gaussian_bp_on_tree}(\texttt{tree}),\ \texttt{beliefs}_0,\ \text{convergence})
\end{aligned}$$

CAS role. The CAS derives the Gaussian BP message equations on the tree structure. Its role is secondary: solving the message-passing update equations once the tree structure is given.

Key discovery bottleneck. The Chow-Liu weight function (mutual information between variable pairs) is the critical choice that cannot be derived from an objective function alone. It requires either (a) the SME to transfer the Chow-Liu optimality result from information theory, or (b) the search engine to evaluate multiple candidate weight functions on the benchmark.

9.3. Case Study 3: BP-Guided Sphere Decoding. BP-guided sphere decoding exemplifies *algorithmic combination*: two existing paradigms (belief propagation and tree search) are composed via type adapters to produce a superior hybrid.

DSL program.

$$\begin{aligned}
&\texttt{beliefs} = \texttt{iterate}(\texttt{gaussian_bp_message},\ \texttt{fg},\ 5\ \text{steps})\\
&\texttt{ordering} = \texttt{beliefs_to_ordering}(\texttt{beliefs})\\
&\hat{\mathbf{x}} = \texttt{best_first_search}(\texttt{tree}(\mathit{sys}, \texttt{ordering}),\ \texttt{beliefs_to_heuristic}(\texttt{beliefs}),\ K)
\end{aligned}$$

Discovery mechanism. The search engine finds this combination by enumerating type-correct paths: `gaussian_bp_message` produces a `GaussianBeliefSet`; the adapter `beliefs_to_ordering` converts it to an `Ordering` required by tree search; the adapter `beliefs_to_heuristic` converts it to a `Heuristic` for pruning. No structural analogy or CAS derivation is needed—pure type-guided composition suffices.

DSL representation of OSIC. As a further combination example, BLAST with MMSE ordering (OSIC) is:

$$\hat{\mathbf{x}} = \texttt{fold_layers}\Big(\texttt{sic_detect_one},\ \texttt{beliefs_to_ordering}(\texttt{estimate_to_beliefs}(\cdot)),\ \mathit{sys}\Big)$$

Table 3: Automated discoverability comparison across the three case studies.

Aspect	**EP-MIMO**	**GTA**	**BP-Guided SD**
Innovation type	Optimisation	Structural	Combination
Primary component	CAS	SME + Search	DSL type search
Key conceptual leap	CAS-derivable	Tree approx. (structural)	Type adapter
CAS role	Central	Minor	None
SME role	None	Central	None
Difficulty	Medium	Medium–High	Low–Medium
Bottleneck	KL dir. + factorisation	Chow-Liu weight fn.	None significant

9.4. Discoverability Comparison. **Key finding.** EP is more derivable under the CAS because its core innovations arise from optimisation decomposition—a purely algebraic process—whereas GTA's tree approximation is a structural selection requiring cross-domain analogical reasoning. BP-guided sphere decoding is the most easily discoverable, requiring only type-guided composition.

10. Discussion: Scope and Limitations

10.1. DSL Granularity Trade-off. The most critical design choice in AlphaDetect is the granularity of the DSL:

- **Too coarse** (only matrix operations): GTA and EP cannot be derived because the factor-graph type does not exist.
- **Too fine** (tree approximation as a primitive): the discovery is trivially guided; the system has been told the answer.
- **Target granularity**: Include the individual components of the solution (factor graph type, spanning tree algorithm, message passing step, moment computation) but not pre-assembled combinations. The "weight function for Chow-Liu" is discovered, not given.

10.2. The Irreducible Role of Structural Intuition. The GTA analysis reveals a fundamental asymmetry: *optimisation-based* innovations (EP, EM, ADMM) are CAS-derivable once the objective and factorisation are chosen, while *structure-based* innovations (tree approximation, lattice representation of MIMO) require cross-domain analogical reasoning. No formal mechanism fully automates structural intuition; SME provides a systematic—but imperfect—surrogate.

10.3. Axiomatic Probing as a Semantic Fallback. When a discovered intermediate object has no type in the DSL (e.g., because it emerged from a neural component), *axiomatic probing* provides a semantic fallback:

- Test whether the object satisfies graph axioms (adjacency, no-cycle) ⇒ assign `Tree` type.
- Test metric axioms (non-negativity, symmetry, triangle inequality) ⇒ `Metric` type.
- Test Gaussian moment conditions ⇒ `GaussianDistribution` type.

Once typed, the object can participate in typed DSL compositions.

11. Related Work

Gentner (1983) [1] Structure Mapping Theory: analogy as relational structure alignment. Foundation for AlphaDetect's cross-domain transfer mechanism.

Falkenhainer, Forbus & Gentner (1989) [2] Structure Mapping Engine: the algorithmic implementation of SMT. AlphaDetect extends SME with LLM-based semantic matching.

Ellis et al. (2021) [3] DreamCoder: library learning by abstracting shared program fragments. AlphaDetect adopts the wake–abstraction–dream cycle for the concept self-enlargement mechanism.

Chalmers, French & Hofstadter (1992) [5] Critique of AI methodology regarding the representation bottleneck in analogy. Directly motivates our LLM-based predicate generation.

Minka (2001) [6] Expectation Propagation for approximate Bayesian inference. The algorithm re-derived in Case Study 1.

Turney (2008) [4] Latent Relation Mapping Engine: SME without hand-coded LISP input, using vector-space representations. Directly relevant to our LLM-embedding extension of SME matching.

12. Summary and Contributions

AlphaDetect addresses the hard problem of *explainable algorithm discovery* for MIMO detection. Its principal contributions are:

1. **Formal characterisation of conceptual innovation.** We identify three forms (structural analogy, optimisation-driven derivation, algorithmic combination) and map each to a tractable automated mechanism.

2. **Typed DSL with algorithmic combinators and adapters.** The DSL provides a type-safe compositional language in which known algorithms can be expressed and new ones searched.

3. **Problem transformation library.** Intermediate variable invention is recast as witness generation from typed transformations, converting an open-ended creative act into structured search.
4. **CAS integration for automatic algorithm derivation.** We show that EP-MIMO and all its variants can be derived by CAS-driven coordinate descent, with cavity and tilted distributions emerging as algebraic by-products.
5. **LLM-augmented Structure Mapping Engine.** LLMs are used as predicate generators and source-domain recommenders—providing semantic bridging—while SME enforces formal structural consistency and generates candidate inferences that drive the search.
6. **Unified system architecture.** The four components (DSL, transformations, CAS, SME) are integrated into a coherent system with MCTS/evolutionary search, complexity evaluation, performance benchmarking, and self-enlarging concept spaces.
7. **Comprehensive case studies.** We demonstrate through EP-MIMO, GTA, and BP-guided sphere decoding that the framework covers all three innovation types, from pure algebraic derivation to cross-domain structural analogy to type-guided algorithmic combination.

References

[1] D. Gentner, "Structure-mapping: A theoretical framework for analogy," *Cognitive Science*, vol. 7, no. 2, pp. 155–170, 1983.

[2] B. Falkenhainer, K. D. Forbus, and D. Gentner, "The structure-mapping engine: Algorithm and examples," *Artificial Intelligence*, vol. 41, no. 1, pp. 1–63, 1989.

[3] K. Ellis, C. Wong, M. Nye, M. Sable-Meyer, L. Cary, L. Morales, L. Hewitt, A. Solar-Lezama, and J. Tenenbaum, "DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning," *Proc. ACM PLDI*, 2021.

[4] P. D. Turney, "The latent relation mapping engine: Algorithm and experiments," *Journal of Artificial Intelligence Research*, vol. 33, pp. 615–655, 2008.

[5] D. J. Chalmers, R. M. French, and D. R. Hofstadter, "High-level perception, representation, and analogy: A critique of artificial intelligence methodology," *Journal of Experimental & Theoretical Artificial Intelligence*, vol. 4, no. 3, pp. 185–211, 1992.

[6] T. P. Minka, "Expectation propagation for approximate Bayesian inference," *Proc. UAI*, pp. 362–369, 2001.

[7] C. Choi and A. Singer, "A tree-reweighted belief propagation algorithm for restoring sparse signals," *Proc. ICASSP*, 2006.

[8] S. Colton, *Automated Theory Formation in Pure Mathematics*. Springer, 2002.